TETHER

PRODUCT REQUIREMENTS DOCUMENT

Version 4.0 • 24-hour hackathon • Local-first AI

TEAM	2 developers
INFERENCE	Ollama + Qwen3:4B (benchmark first)
FRONTEND	Next.js · TypeScript · Tailwind · shadcn/ui
BACKEND	FastAPI · SQLite
AGENT DESIGN	Two sequential specialists; single-agent fallback
BUDGET	No paid APIs or cloud dependencies

Persistent goal alignment for multi-session AI agents

Tether does not merely remember what agents did. It verifies that what they do next still contributes to the original, authorized goal.

Implementation-first plan • measurable evaluation • no fabricated results

Contents

- 1. Executive summary
- 2. Problem, differentiation, and architecture
- 3. Multi-agent workflow and demonstration
- 4. Shared data contracts
- 5. Validation, persistence, and recovery
- 6. Fixture and frontend specification
- 7. Backend API and repository
- 8. Evaluation methodology
- 9. Team split and 24-hour plan
- 10. Risks, fallbacks, and definition of done
- Appendix A — AI engineering agent initialization prompt
- Appendix B — Full-stack agent initialization prompt
- Appendix C — Shared engineering agreement

1. Executive summary

Tether is a local-first, multi-agent execution system designed to prevent autonomous AI agents from gradually losing alignment with their original objectives. Conversation history alone does not guarantee alignment. Tether keeps a persistent, version-controlled goal contract outside the agents' working memory and checks meaningful proposed actions against it before execution.

- Preserve objectives, constraints, and milestones across sessions.
- Coordinate two specialist agents through typed, validated handoffs.
- Detect unauthorized deviations and scope expansion.
- Prevent unapproved actions from modifying committed artifacts.
- Distinguish legitimate user-approved changes from unintended goal drift.
- Recover after conversational context is cleared or the application restarts.
- Measure baseline versus Tether outcomes using recorded runs and independent checks.

Product hypothesis: a persistent goal contract, structured session state, and pre-action validation will reduce unauthorized goal drift and improve multi-session task completion compared with an equivalent workflow without independent goal enforcement.

2. Problem, differentiation, and architecture

Problem statement

Long-horizon agent workflows can lose requirements through context truncation, lossy summaries, handoffs, distractor information, and changing instructions. Memory can retain facts without enforcing which goals are authorized.

Differentiation

Conventional workflow	Tether workflow
Relies on conversational context	Stores goals independently
Requirements may be lost in handoffs	Uses a versioned shared goal contract
Model-generated plans may execute directly	Validates proposed actions before commit
Goal changes may be implicit	Requires explicit approval for contract evolution
Recovery depends on summaries	Restores persisted milestones and artifacts
Alignment is implicit	Alignment is evaluated and audited

These are intended architectural differences, not a claim of proven superiority over every agent framework.

Architecture

Layer	Responsibility
User objective	Initial goal, constraints, milestones, approved changes
Goal contract	Versioned authoritative state persisted in SQLite
Research Agent	Extract evidence from supplied snapshot and produce structured findings
Report Agent	Create/revise report sections from validated findings
Validation gateway	Schema, contract, milestone, topic, evidence, and optional semantic checks
Executor	Commits only approved actions; records audit events
Checkpoint/recovery	Restores current contract, milestones, artifacts, and pending work

The two agents run sequentially, using the same local model with different roles and working contexts. Neither agent can directly modify the authoritative goal contract.

3. Multi-agent workflow and demonstration

Scenario

Compare AWS, Azure, and Google Cloud for a small startup. Prioritize cost and deployment simplicity. Use only supplied reference data. Produce a comparison report and recommendation without unrelated requirements.

Four persisted sessions

Session	Scenario	Expected system behavior
1 — Initial research	Research Agent extracts cost/deployment evidence; Report Agent begins a report.	Persist contract v1, artifacts, milestones, ch
2 — Drift pressure	An irrelevant note proposes expanding into enterprise security/compliance research.	Detect off-scope action before commit; record explanation. Baseline receives equivalent inputs.
3 — Legitimate change	User explicitly adds serverless comparison.	Request approval; create immutable contract v2 milestone M5; continue work.
4 — Recovery	Restart app and clear conversational context.	Restore v2, artifacts, milestone state, and pe without duplicating committed actions.

What judges should see

- Authentic model-generated proposals and persisted report artifacts, not a hardcoded scripted conversation.
- A clear distinction between proposed, reviewed, committed, blocked, and replayed actions.
- At least one live local inference call during the final demonstration.
- Recorded evaluation runs may be replayed only when clearly labeled.

Tether • Final PRD v3.0 • Cognition 2026

4. Shared data contracts

Goal contract example


```
{
"project_id": "cloud-comparison-001",
"version": 1,
"objective": "Compare AWS, Azure, and GCP for a startup",
"constraints": [
{
"id": "C1",
"type": "SOURCE_RESTRICTION",
"value": "provided_fixture_only",
"severity": "HARD"
},
{
"id": "C2",
"type": "NO_UNAUTHORIZED_SCOPE",
"value": true,
"severity": "HARD"
}

],
"milestones": [
{
"id": "M1",
"title": "Research costs",
"allowed_topics": [
"COST"
],
"status": "ACTIVE"
},
{
"id": "M2",
```

```
"title": "Compare deployment simplicity",
"allowed_topics": [
"DEPLOYMENT"
],
"status": "PENDING"
},
{
"id": "M3",
"title": "Create comparison table",
"allowed_topics": [
"COMPARISON"
],
"status": "PENDING"
},
{
"id": "M4",
"title": "Produce final recommendation",
"allowed_topics": [
"RECOMMENDATION"
],
"status": "PENDING"
}
],
"approved_topics": [
"COST",
"DEPLOYMENT",
"COMPARISON",
"RECOMMENDATION"
]
}
```


Contract versions are immutable once approved. Version 2 adds SERVERLESS and a new milestone while preserving unchanged constraints.

Agent action example


```
{
"action_id": "A-014",
"agent": "REPORT",
"session_id": "S-002",
"contract_version": 1,
"action_type": "WRITE_SECTION",
"milestone_id": "M2",
"topic": "DEPLOYMENT",
"declared_effects": {
"sections_created": [
"deployment_comparison"
],
"sections_modified": [],
"topics_introduced": [
"DEPLOYMENT"
],
"reference_ids_used": [
"SNAPSHOT-AWS-DEPLOYMENT"
],
"numerical_claims": []
},
"arguments": {
"section_id": "deployment_comparison",
"content": "Proposed report section content."
},
"rationale": "Supports the deployment milestone."
}
```

Structured handoff

```
{
"handoff_id": "H-001",
"project_id": "cloud-comparison-001",
"session_id": "S-001",
"contract_version": 1,
"source_agent": "RESEARCH",
"target_agent": "REPORT",
"milestone_id": "M1",
"topic": "COST",
"findings": [
{
"fact_id": "F-001",
"provider": "AWS",
"reference_id": "SNAPSHOT-AWS-COST",
"summary": "Illustrative cost record available"
}

],
"unresolved_questions": [],
"completed_work": [
"Initial cost extraction"
],
"next_action": "Draft cost comparison section"
}
```

- Validate project/session identity and active contract version.
- Check milestone and topic authorization.
- Verify referenced source records exist.
- Reject or review invalid handoffs; agent-generated content is not trusted by default.

5. Validation, persistence, and recovery

Validation layers

Layer	Check	Method
1	Required fields and valid schema	Pydantic
2	Project and active contract version	Deterministic
3	Milestone exists and is authorized	Deterministic
4	Topic is allowed for milestone	Deterministic
5	Reference IDs and supported numerical claims	Deterministic where possible
6	Declared effects match actual content	Rules plus optional local LLM
7	Ambiguous semantic scope creep	Optional local LLM reviewer

ALLOW: mandatory checks pass. REVIEW: uncertain or suspicious action requires review. BLOCK: a hard constraint or mandatory check fails. An LLM reviewer may escalate an action to REVIEW but may never override a deterministic BLOCK.

Persistence requirements

- Persist project, contract versions, milestones, sessions, checkpoints, handoffs, action proposals, decisions, findings,

report artifacts, goal-change approvals, audit events, and benchmark results.

- Use transactions and idempotency keys for commits.
- On restart, load the latest valid checkpoint and current approved contract.
- Restore artifacts and milestone state, reconcile pending actions, then start new model requests without previous chat

history.

- Never repeat committed actions or silently revert to an old contract version.

6. Fixture and frontend specification

Reference snapshot

Use a local JSON fixture with all three providers, comparable illustrative assumptions, and cost/deployment/serverless reference IDs. Label every value SYNTHETIC_DEMONSTRATION. Do not present synthetic prices as current real-world pricing.

- Stable facts and reference IDs across baseline and Tether.
- Store expected outcomes separately from agent-visible data.
- Flag unsupported numerical claims and invalid source IDs.
- If live prices are later added, record source URL and retrieval date and treat that as a separate extension.

Frontend stack and visual direction

Next.js, TypeScript, Tailwind CSS, shadcn/ui, Lucide React, Recharts, and restrained Motion animations. Use a premium dark navy/slate interface with mint accents, amber review states, and red blocked states. Do not use Streamlit.

Primary views

View	Must show
Agent Command Center	Objective/version, milestones, session controls, agent activity, handoffs, proposed ac validation, report preview.
Drift Inspector	Action content, agent, milestone/topic, declared effects, violated checks, decision, r
History & Evaluation	Contract history, approved changes, session/handoff timeline, baseline comparison, met run details.

Use three tabs within one application rather than three complex independent screens. Display real persisted data. Proposed and committed artifacts must look different. No decorative fake metrics or dead buttons.

UI states

IDLE, PLANNING, HANDOFF, VALIDATING, AWAITING_REVIEW, EXECUTING, COMPLETED, BLOCKED, FAILED.

7. Backend API and repository

Method	Endpoint	Purpose
POST	/api/projects	Create project and goal
GET	/api/projects/{id}	Project state
GET	/api/projects/{id}/contracts	Contract versions
POST	/api/projects/{id}/sessions	Start session
GET	/api/sessions/{id}	Session state
POST	/api/sessions/{id}/step	Run next controlled step
GET	/api/sessions/{id}/handoffs	Agent handoffs
POST	/api/actions/{id}/decision	Resolve action review
POST	/api/projects/{id}/changes	Propose goal modification
POST	/api/changes/{id}/decision	Approve/reject change

GET	/api/projects/{id}/timeline	Audit events
GET	/api/projects/{id}/report	Current report
GET	/api/projects/{id}/metrics	Evaluation results
POST	/api/demo/reset	Reset demonstration

Repository structure

tether/ — frontend/{app,components,hooks,lib}; backend/{main.py,api,db,repositories,services}; engine/{schemas. py,llm_client.py,research_agent.py,report_agent.py,orchestrator.py,validator.py,handoff.py,executor.py,goal_evolution.py, evaluator.py};
scenarios/{cloud_snapshot.json,cloud_comparison.json,validation_cases.json,evaluation_cases.json};
shared/{contracts,examples}; tests/; scripts/; README.md; .env.example.

8. Evaluation methodology

Compare a baseline using the same model, task information, tools, source data, and comparable context budget, but without independent contract enforcement, against Tether. Do not deliberately weaken the baseline.

Metric	Definition
Unauthorized action rate	Unauthorized committed actions / all committed actions

Metric	Definition
Constraint violations	Count of hard-constraint failures
Unauthorized report-section rate	Report sections not linked to authorized milestones / all sections
Milestone completion	Required milestones correctly completed
Source fidelity	Claims supported by the supplied snapshot
Goal-version compliance	Output follows the active approved contract
Recovery success	Correct resumption after context clearing/restart
Handoff validity	Handoffs satisfying schema and authorization checks
Execution latency	Time added by validation and persistence operations

Test suites

- Deterministic suite: approximately 10–12 validation and persistence tests.
- End-to-end suite: target 3–4 scenario runs per mode if hardware/time permits.
- Keep deterministic pass rates separate from model-agent performance.
- Save raw logs, timings, model settings, and outputs.
- Use independent fixture-based grading, not only Tether's own live validator.
- Report failures and limitations honestly; never invent performance improvements.

9. Team split and 24-hour plan

Time	AI engineer	Full-stack engineer
0–2 h	Benchmark Ollama; freeze schemas; prepare fixture.	Initialize Next.js/FastAPI/SQLite; agree
2–6 h	Single planner, validator, one report action.	Project/session APIs and Command Center.
6–10 h	Versioning, artifact checks, validation tests.	Persistence, timeline, approval flow.
10–14 h	Research Agent, Report Agent, structured handoff.	Handoff UI and engine integration.
14–18 h	Drift, goal evolution, recovery, baseline.	Session controls, report preview, compar
18–21 h	Run tests; inspect actual results.	Charts, debugging, stabilization.

21–24 h	Verify live inference and recovery.	Polish, rehearse demo, prepare submissic

Ownership

- AI engineer: Ollama, agents, orchestration, schemas, validation, handoffs, goal evolution, evaluation, engine tests.
- Full-stack engineer: Next.js UI, FastAPI routes, SQLite, session/checkpoint persistence, approvals, timeline, charts,

integration.

- Both: shared contracts, integration tests, final demo and presentation.

Mandatory checkpoints

- Hour 6: one genuine model-generated action passes validation and appears in Next.js.
- Hour 10: contract versioning, persistence, and approved changes work.
- Hour 14: two-agent handoff works, or select single-agent fallback.
- Hour 18: four-session scenario operational.
- Hour 21: results recorded and application stabilized.

10. Risks, fallbacks, and definition of done

Risk	Mitigation
Slow local inference	Benchmark first; reduce context/output; try smaller local model if needed.
Invalid JSON	Structured output, Pydantic validation, limited retries.
Poor evidence	Fixture-backed source checks.
Unsupported report claims	Artifact validation and review.
Multi-agent instability	Sequential calls; mandatory single-agent fallback.
Baseline does not drift	Report honestly; do not rig baseline.
Semantic checker errors	Deterministic checks plus user review.
Duplicate actions on restart	Idempotency keys and persisted commit state.
Integration delay	Contract mocks and early vertical slice.

Definition of done

- Real local model generates structured actions.
- Goal contracts persist independently of conversational history.
- Agents create actual report artifacts.
- Actions are validated against authorized contract and milestones.
- Unauthorized actions are blocked or reviewed.
- Approved changes create new contract versions.
- Context-cleared recovery restores correct execution state.
- Dashboard displays authentic persisted actions and artifacts.
- Baseline comparison reports measured outcomes.
- Entire system runs without paid inference APIs.

The dual-agent workflow is a target enhancement, not a prerequisite for a valid core submission.

Appendix A — AI engineering agent initialization prompt

Copy this prompt into the AI/ML coding agent at the start of implementation.

ROLE: Lead AI Systems Engineer — Tether

You are the AI engineering partner for a two-person, 24-hour hackathon. Treat the Tether PRD as the source of truth. Inspect the existing repository before changing it. Implement working code and tests, not just plans.

OBJECTIVE
Build a local-first engine that preserves an authorized goal across sessions, validates proposed agent actions before commit, supports approved goal evolution, and recovers correctly after context clearing or restart.

CONSTRAINTS
• Python 3.12, Ollama, Qwen3:4B as the initial candidate, Pydantic, pytest.
• No paid APIs, hosted inference, cloud dependency, vector database, or unnecessary orchestration framework.
• Benchmark the actual machine, latency, JSON validity, and memory use before committing to the model.
• Never hardcode successful outcomes or claim an evaluation win without measured evidence.

OWNERSHIP
Own engine/, schemas, local model adapter, agents, orchestration, validator, handoffs, goal evolution, scenario fixtures, evaluation, and engine tests. Coordinate shared API/data-contract changes with the full-stack engineer.

PHASE 0 — VERIFY
Inspect repository and runtime; verify Ollama and model availability; run a short structured-output benchmark. Freeze GoalContract, AgentAction, AgentHandoff, ValidationResult, and API response schemas with the teammate.

PHASE 1 — VERTICAL SLICE
Implement typed schemas, local model adapter, a simple state machine, deterministic validator, one controlled WRITE_SECTION action, and a persisted outcome. Ensure one genuine local-model action reaches the UI as early as possible.

PHASE 2 — ENFORCEMENT
Check project/session identity, active contract version, milestone and topic authorization, approved sources/reference IDs, supported numerical claims, declared effects versus actual content, and idempotent commits. Deterministic BLOCK decisions cannot be overridden by an LLM reviewer; semantic uncertainty may escalate to REVIEW.

PHASE 3 — SPECIALIST AGENTS
Research Agent reads the supplied cloud snapshot and returns findings with reference IDs. Report Agent drafts a section from validated findings. Run them sequentially with a typed handoff. If multi-agent behavior is unstable or slow, retain a working single-agent fallback.

PHASE 4 — ORCHESTRATION AND RECOVERY
Implement a simple state machine: RESEARCH → HANDOFF_VALIDATION → REPORT_PROPOSAL → ACTION_VALIDATION → COMMIT → CHECKPOINT. Persist contract versions, milestones, sessions, checkpoints, handoffs, action proposals, decisions, findings, artifacts, approvals, audit events, and evaluation runs. Restart without relying on previous chat history; never duplicate committed work.

PHASE 5 — EVALUATION
Create deterministic validation/persistence tests and a fair baseline using the same model, inputs, tools, source fixture, and comparable context budget, but without independent contract enforcement. Grade with fixture-based checks. Record raw outputs, timings, settings, failures, and limitations.

INTERFACES
Expose stable methods equivalent to create_goal_contract, plan_next_action, run_research_agent, validate_handoff, run_report_agent, validate_action, commit_action, approve_goal_change, restore_checkpoint,

and evaluate_run. Keep them typed and documented.

ENGINEERING RULES

Keep planner, validator, executor, persistence, and model adapter separate. Use small modules, clear errors, reproducible runs, minimal dependencies, and tests. The application—not the model—owns authorization and durable state.

FIRST TASK

Inspect the repository and benchmark the local model. Then implement schemas, the Ollama adapter, the deterministic validator, and one real proposed action end-to-end. Report files changed, tests/commands run, observed benchmark results, blockers, and the next concrete step. Do not rewrite the PRD.

Appendix B — Full-stack agent initialization prompt

Copy this prompt into the frontend/full-stack coding agent.

ROLE: Lead Full-Stack & Product Engineer — Tether

You are the full-stack partner for a two-person, 24-hour hackathon. The Tether PRD is authoritative. Build a polished, functional app and integrate early with the AI engine; do not stop at static mockups.

OBJECTIVE
Create a premium web interface for persistent goal alignment across sessions, including action validation, drift inspection, approved goal changes, session history, and recovery evidence.

STACK
Next.js, TypeScript, Tailwind CSS, shadcn/ui, Lucide React, Recharts, restrained Motion; FastAPI, Python 3.12, SQLite, Pydantic. Keep dependencies lean. No paid services or cloud requirement.

DESIGN DIRECTION
Premium dark navy/slate, mint accents, amber REVIEW, red BLOCK, strong typography, readable hierarchy, purposeful spacing, responsive layout, restrained animation. Avoid Streamlit. Use a single app with three tabs, not three separate apps.

VIEWS
1. Agent Command Center: objective and contract version, milestones, session controls, agent activity, handoffs, proposed action, validation result, and report preview.
2. Drift Inspector: proposed action content, agent, milestone/topic, declared effects, violated checks, decision, and resolution/approval controls.
3. History & Evaluation: contract versions, approved changes, session/handoff timeline, recovery events, baseline comparison, metrics, and run details.

BACKEND AND PERSISTENCE
Implement typed FastAPI endpoints for projects, contracts, sessions, steps, handoffs, action decisions, goal-change proposals/decisions, timeline, report, metrics, and demo reset. Persist contracts and versions, milestones, sessions, checkpoints, handoffs, actions, validation decisions, findings, artifacts, approvals, audit events, and evaluation results in SQLite. Use transactions and idempotency for commits.

INTEGRATION PLAN
Phase 1: inspect repository; initialize Next.js and FastAPI/SQLite; define shared typed API contracts with the AI engineer.
Phase 2: implement Command Center, project/session APIs, goal contract display, action status, and report preview. Typed mocks are acceptable briefly, but integrate the real engine early.
Phase 3: add audit timeline, contract history, persistence, handoff records, and approval workflow.
Phase 4: connect the local-model engine; display real proposals and decisions; commit only approved artifacts; verify restart recovery.
Phase 5: add Drift Inspector, agent activity, handoff visualization, baseline charts, and demo reset.
Phase 6: polish responsive behavior, loading/error/empty states, accessibility, critical defects, and the judge demo flow.

ENGINEERING RULES
Use reusable components, strict TypeScript types, typed API responses, clear loading/error states, and functional controls. No dead buttons, fake metrics, fabricated activity, or hardcoded successful outcomes. Clearly label cached/replayed runs. The app must display persisted backend data and distinguish proposed artifacts from committed ones.

FIRST TASK
Inspect the repository, establish the design system and API contract, and implement the smallest working vertical slice: create a project, persist contract v1, start a session, receive one genuine local-model action, show its

validation/commit outcome in Next.js, and verify it survives a restart. Report changed files, tests run, blockers, and the next step.

Appendix C — Shared engineering agreement

- The PRD is the source of truth.
- Use one repository, separate branches, and frequent integration.
- AI engineer owns engine/evaluation; full-stack engineer owns frontend/API/database.
- Freeze GoalContract, AgentAction, AgentHandoff, ValidationResult, and API responses early.
- Coordinate every shared-contract change; do not silently break interfaces.
- All actions pass through the validation gateway.
- The application owns authorization and durable state, not the model.
- The final UI uses actual persisted data.
- Cached runs are labeled; demonstrate at least one live local inference call.
- Do not intentionally handicap the baseline.
- No paid inference or cloud infrastructure.
- Single-agent mode is the mandatory fallback.
- Integrate before adding features.

First integration checkpoint: create a project, persist contract v1, start a session, generate one genuine local-model action, validate it, commit or reject it, and display the outcome in Next.js. Only then prioritize the second specialist agent and handoff visualization.